
A Bunched Approach to Directed HoTT

Louise Leclerc

Samuel Mimram

September 17, 2026

Outline

- ① Homotopy type theory
- ② Simplicial type theory
- ③ The problem of hom types
- ④ The bunched solution
- ⑤ From theory to application

Presentation

(Homotopy) Type Theory:

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...
- Suitable for formalizing mathematics / homotopy theory

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...
- Suitable for formalizing mathematics / homotopy theory
- **Goal:** replace spaces by *directed spaces*

Intuitionistic Type Theory

MLTT:

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- Statements have the form $\Gamma \vdash a : A$ or $\Gamma \vdash a = b : A$ where $\Gamma = (x_1 : A_1, x_2 : A_2, \dots, x_n : A_n)$.

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- Statements have the form $\Gamma \vdash a : A$ or $\Gamma \vdash a = b : A$ where $\Gamma = (x_1 : A_1, x_2 : A_2, \dots, x_n : A_n)$.
- Basic types: $\mathbb{0}$, $\mathbb{1}$, **Bool**, $\mathbb{N}$, ...

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- Statements have the form $\Gamma \vdash a : A$ or $\Gamma \vdash a = b : A$ where $\Gamma = (x_1 : A_1, x_2 : A_2, \dots, x_n : A_n)$.
- Basic types: $\mathbb{0}$, $\mathbb{1}$, **Bool**, $\mathbb{N}$, $\dots$
- Universes $\mathcal{U}$ closed under $\rightarrow$, $\times$, $+$, $\dots$

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- Statements have the form $\Gamma \vdash a : A$ or $\Gamma \vdash a = b : A$ where $\Gamma = (x_1 : A_1, x_2 : A_2, \dots, x_n : A_n)$.
- Basic types: $\mathbb{0}$, $\mathbb{1}$, **Bool**, $\mathbb{N}$, $\dots$
- Universes $\mathcal{U}$ closed under $\rightarrow$, $\times$, $+$, $\dots$
- $\mathbb{0} : \mathcal{U}$, $\mathbb{N} \times \mathbb{1} : \mathcal{U}$, $\mathbb{N} \rightarrow \mathbf{Bool} : \mathcal{U} \dots$

More type formers

- Π -types:

$$A : \mathcal{U}, B : A \rightarrow \mathcal{U} \quad \rightsquigarrow \quad \prod_{x:A} B(x) : \mathcal{U}.$$

$$f : \prod_{x:A} B(x), a : A \quad \rightsquigarrow \quad f(a) : B(a).$$

More type formers

- Π -types:

$$A : \mathcal{U}, B : A \rightarrow \mathcal{U} \quad \rightsquigarrow \quad \prod_{x:A} B(x) : \mathcal{U}.$$

$$f : \prod_{x:A} B(x), a : A \quad \rightsquigarrow \quad f(a) : B(a).$$

- Σ -types:

$$A : \mathcal{U}, B : A \rightarrow \mathcal{U} \quad \rightsquigarrow \quad \sum_{x:A} B(x) : \mathcal{U}.$$

$$a : A, u : B(a) \quad \rightsquigarrow \quad (a, u) : \sum_{x:A} B(x).$$

Curry–Howard

Correspondance between types and logical propositions.

- $\rightarrow$ is the implication arrow.
- $\times$ is the conjunction.
- $+$ is a constructive disjunction.
- $\mathbb{0}$ is false, $\mathbb{1}$ is true.

Logic $\longleftrightarrow$ Algebra

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$
- **induction principle**: to prove something for every $p : x = y$, it suffices to treat the case $y \equiv x$, $p \equiv \text{refl}_x$.

The identity type

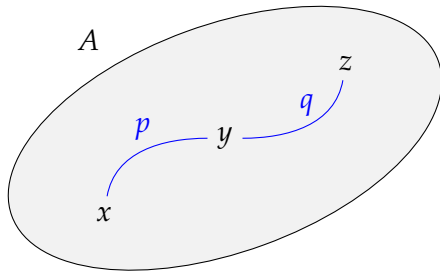
- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$
- **induction principle**: to prove something for every $p : x = y$, it suffices to treat the case $y \equiv x$, $p \equiv \text{refl}_x$.
- Think of $x = y$ as the type of paths from x to y .

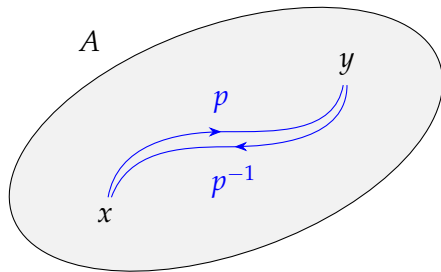
Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$



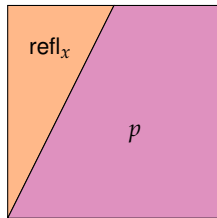
Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$



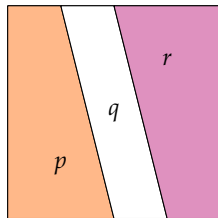
Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$
- **unit:** $\text{refl}_x \cdot p =_{x=y} p$ and $p \cdot \text{refl}_y =_{x=y} p$



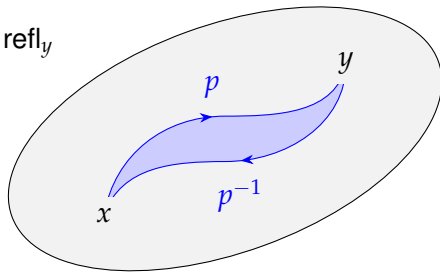
Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$
- **unit:** $\text{refl}_x \cdot p =_{x=y} p$ and $p \cdot \text{refl}_y =_{x=y} p$
- **associativity:** $p \cdot (q \cdot r) =_{x=y} (p \cdot q) \cdot r$



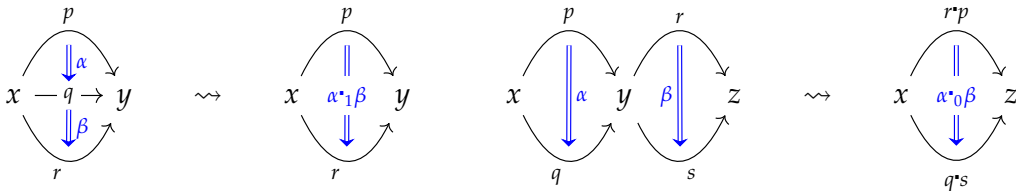
Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$
- **unit:** $\text{refl}_x \cdot p =_{x=y} p$ and $p \cdot \text{refl}_y =_{x=y} p$
- **associativity:** $p \cdot (q \cdot r) =_{x=y} (p \cdot q) \cdot r$
- **inversion:** $p \cdot p^{-1} =_{x=y} \text{refl}_x$ and $p^{-1} \cdot p =_{x=y} \text{refl}_y$



Groupoid structure on types

- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$
- **unit:** $\text{refl}_x \cdot p =_{x=y} p$ and $p \cdot \text{refl}_y =_{x=y} p$
- **associativity:** $p \cdot (q \cdot r) =_{x=y} (p \cdot q) \cdot r$
- **inversion:** $p \cdot p^{-1} =_{x=y} \text{refl}_x$ and $p^{-1} \cdot p =_{x=y} \text{refl}_y$
- **composition of higher cells:** $\alpha : p =_{x=y} q, \beta : q =_{x=y} r \rightsquigarrow \alpha \cdot \beta : p =_{x=y} r \dots$



Groupoid structure on types

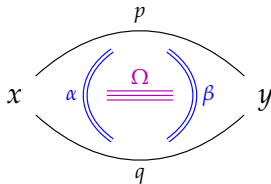
- **concatenation:** $p : x =_A y, q : y =_A z \rightsquigarrow p \cdot q : x =_A z$
- **inverse:** $p : x =_A y \rightsquigarrow p^{-1} : y =_A x$
- **unit:** $\text{refl}_x \cdot p =_{x=y} p$ and $p \cdot \text{refl}_y =_{x=y} p$
- **associativity:** $p \cdot (q \cdot r) =_{x=y} (p \cdot q) \cdot r$
- **inversion:** $p \cdot p^{-1} =_{x=y} \text{refl}_x$ and $p^{-1} \cdot p =_{x=y} \text{refl}_y$
- **composition of higher cells:** $\alpha : p =_{x=y} q, \beta : q =_{x=y} r \rightsquigarrow \alpha \cdot \beta : p =_{x=y} r \dots$

Theorem

Every type is equipped with a canonical structure of ∞ -groupoid.

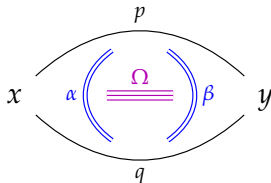
∞ -groupoids

Spaces up to homotopy



∞ -groupoids

Spaces up to homotopy



$$x, y : A$$

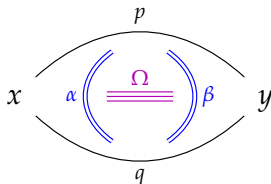
$$p, q : x =_A y$$

$$\alpha, \beta : p =_x =_A y q$$

$$\Omega : \alpha =_p =_x =_A \ y \ q \ \beta$$

∞ -groupoids

Spaces up to homotopy



$$x, y : A \qquad p, q : x =_A y \qquad \alpha, \beta : p =_{x =_A y} q \qquad \Omega : \alpha =_{p =_{x =_A y} q} \beta$$

Formalisation of homotopy theory

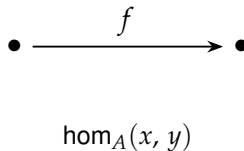
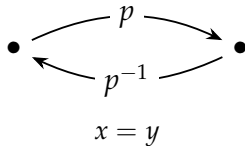
(Feudenthal, long exact sequences, sphere homotopy groups...).

Non-directedness of HoTT

- Paths $x = y$ are always *invertible*.

Non-directedness of HoTT

- Paths $x = y$ are always *invertible*.

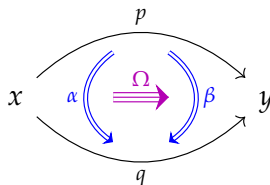


∞ -categories

"Directed" spaces up to homotopy

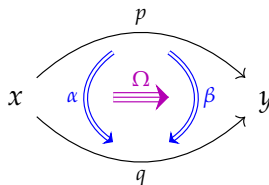
∞ -categories

"Directed" spaces up to homotopy



∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

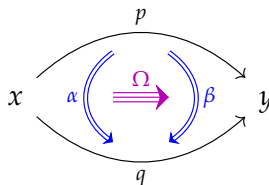
$$p, q : \text{hom}_A(x, y)$$

$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

$$p, q : \text{hom}_A(x, y)$$

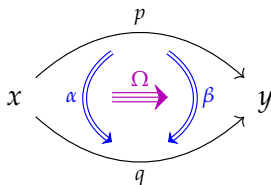
$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

Rewriting, concurrency,

∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

$$p, q : \text{hom}_A(x, y)$$

$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

Rewriting, concurrency,
synthetic manipulation of higher categories.

Simplicial type theory

- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz...

Simplicial type theory

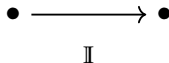
- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz...
- Postulate a formal interval $(\mathbb{I}, 0, 1, \leq)$.

Simplicial type theory

- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz...
- Postulate a formal interval $(\mathbb{I}, 0, 1, \leq)$.
- Goal: define $(\infty, 1)$ -categories internally.

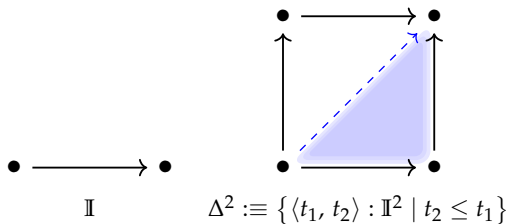
The interval

We add to the syntax an interval $(\mathbb{I}, 0, 1, \leq)$



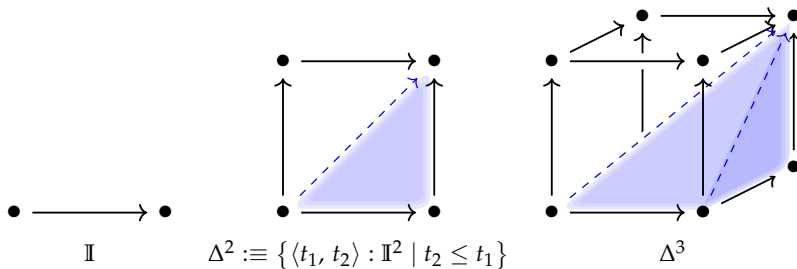
The interval

We add to the syntax an interval $(\mathbb{I}, 0, 1, \leq)$



The interval

We add to the syntax an interval $(\mathbb{I}, 0, 1, \leq)$



Simplicial types

X type

 $\mathbb{I} \rightarrow X$ type

Simplicial types

X type

$\mathbb{I} \rightarrow X$ type

$$\mathrm{hom}_X(x, y) :\equiv \sum_{f:\mathbb{I} \rightarrow X} (f(0) = x) \times (f(1) = y)$$

Simplicial types

X type

$\mathbb{I} \rightarrow X$ type

$$\mathrm{hom}_X(x, y) :\equiv \sum_{f:\mathbb{I} \rightarrow X} (f(0) = x) \times (f(1) = y)$$

but also

$\Delta^2 \rightarrow X$ type

$\Delta^3 \rightarrow X$ type

$\vdots$

Simplicial types

X type

$\mathbb{I} \rightarrow X$ type

$$\mathrm{hom}_X(x, y) :\equiv \sum_{f:\mathbb{I} \rightarrow X} (f(0) = x) \times (f(1) = y)$$

but also

$\Delta^2 \rightarrow X$ type

$\Delta^3 \rightarrow X$ type

$\vdots$

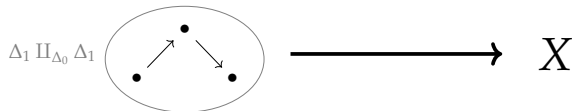
So each type give rise to a *simplicial type*.

Categories (1)

When does X behaves as a category ?

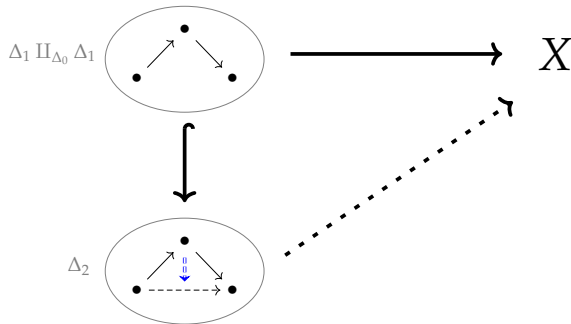
Categories (1)

When does X behaves as a category ?



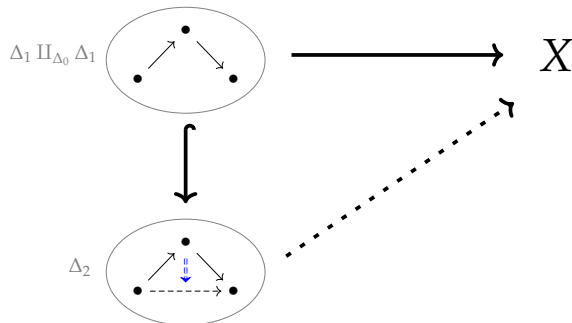
Categories (1)

When does X behaves as a category ?



Categories (1)

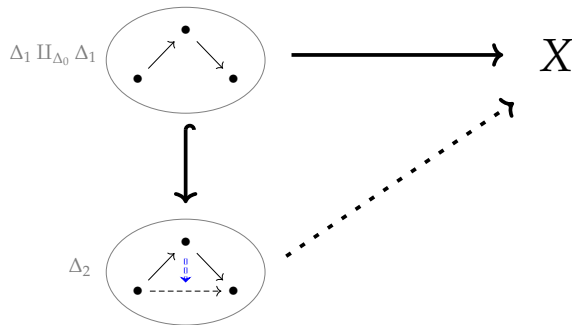
When does X behaves as a category ?



$$(\Delta^2 \rightarrow X) \xrightarrow{\sim} (\mathbb{I} +_{\mathbb{1}} \mathbb{I} \rightarrow X)$$

Categories (1)

When does X behaves as a category ?

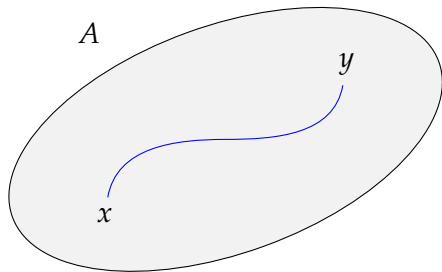


$$(\Delta^2 \rightarrow X) \xrightarrow{\sim} (\mathbb{I} +_{\mathbb{1}} \mathbb{I} \rightarrow X)$$

$$\sum_{x,y,z:X} \text{hom}_X(x, y) \times \text{hom}_X(y, z) \xrightarrow{\circ} \text{hom}_X(x, z)$$

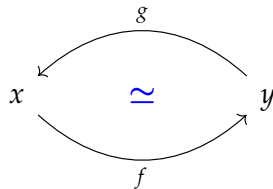
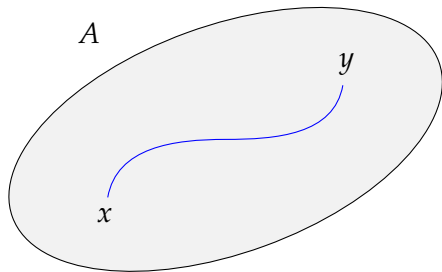
Categories (2)

A type A as before admits composites $g \circ f$.



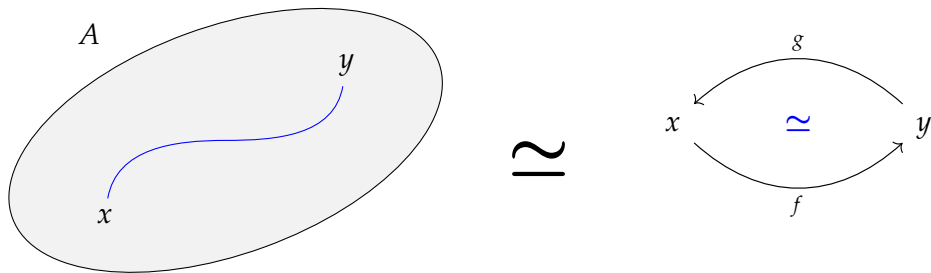
Categories (2)

A type A as before admits composites $g \circ f$.



Categories (2)

A type A as before admits composites $g \circ f$.



$$(x =_A y) \simeq \text{iso}_A(x, y)$$

Hom Types in STT

Type of 1-cells = $\mathbb{I} \rightarrow A$.

$$x \longrightarrow y$$

Hom Types in STT

Type of 1-cells = $\mathbb{I} \rightarrow A$.

$$x \longrightarrow y$$

Cells mediating between 1-cells should be 2-cells.

Hom Types in STT

Type of 1-cells = $\mathbb{I} \rightarrow A$.

$$x \longrightarrow y$$

Cells mediating between 1-cells should be 2-cells.

Type of 2-cells = $\mathbb{I} \rightarrow (\mathbb{I} \rightarrow A) \simeq \mathbb{I}^2 \rightarrow A$.

$$\begin{array}{ccc} x & \longrightarrow & y \\ \downarrow & & \downarrow \\ x' & \longrightarrow & y' \end{array}$$

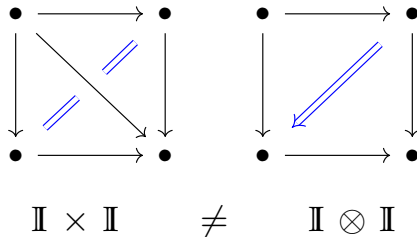
The problem of hom-types

x, y invertible in $\mathbb{I}$ $\Rightarrow$ (x, y) invertible in $\mathbb{I}^2$.

The problem of hom-types

x, y invertible in $\mathbb{I} \Rightarrow (x, y)$ invertible in $\mathbb{I}^2$.

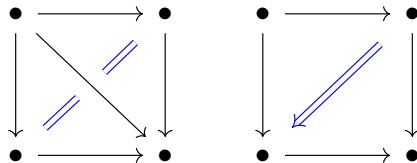
$\mathbb{I}^2$ is 1-categorical: hom of a hom can only be a *groupoid* again.



The problem of hom-types

x, y invertible in $\mathbb{I}$ $\Rightarrow$ (x, y) invertible in $\mathbb{I}^2$.

$\mathbb{I}^2$ is 1-categorical: hom of a hom can only be a *groupoid* again.



$$\mathbb{I} \times \mathbb{I} \neq \mathbb{I} \otimes \mathbb{I}$$

$$\mathrm{hom}_A(x, y) \neq \sum_{f: \mathbb{I} \rightarrow A} (f \cdot 0 = x) \times (f \cdot 1 = y)$$

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \multimap -)$

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \multimap -)$

$$\mathbb{I} \multimap (\mathbb{I} \multimap A) \simeq (\mathbb{I} \otimes \mathbb{I} \multimap A)$$

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \multimap -)$

$$\mathbb{I} \multimap (\mathbb{I} \multimap A) \simeq (\mathbb{I} \otimes \mathbb{I} \multimap A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \multimap -)$

$$\mathbb{I} \multimap (\mathbb{I} \multimap A) \simeq (\mathbb{I} \otimes \mathbb{I} \multimap A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$

$$\mathbb{I} \rightarrow (\mathbb{I} \rightarrow A) \simeq (\mathbb{I} \otimes \mathbb{I} \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$

$$\mathbb{I} \rightarrow (\mathbb{I} \rightarrow A) \simeq (\mathbb{I} \otimes \mathbb{I} \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

$$\bullet \xRightarrow{\quad} \bullet$$

$$\Gamma \otimes \Delta$$

$$\bullet \text{ — } \bullet$$

$$\Gamma, \Delta$$

Replacing $\rightarrow$

- Idea: replacing $(- \times \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$ by $(- \otimes \mathbb{I}) \dashv (\mathbb{I} \rightarrow -)$

$$\mathbb{I} \rightarrow (\mathbb{I} \rightarrow A) \simeq (\mathbb{I} \otimes \mathbb{I} \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

$$\bullet \xrightarrow{\quad} \bullet$$

$$\Gamma \otimes \Delta$$

$$\bullet \text{ — } \bullet$$

$$\Gamma, \Delta$$

$$\Gamma \otimes \Delta \neq \Delta \otimes \Gamma$$

Interpreted as the Crans-Gray tensor product.

Bunches

$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Bunches

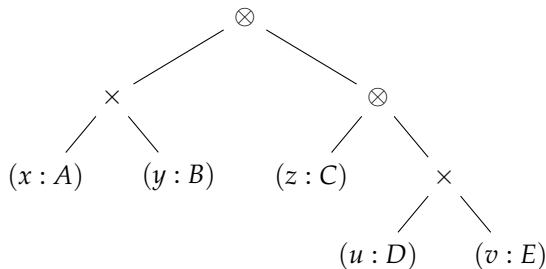
$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Judgments are derived in a bunch Γ , which has a *tree* structure.

Bunches

$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Judgments are derived in a bunch Γ , which has a *tree* structure.



$$\vdash y \otimes (u, v) : B \otimes (D \times E)$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash f(a) : B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash f(a) : B}$$

$$(\multimap\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \rho x. b : A \multimap B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash f(a) : B}$$

$$(\multimap\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \wp x. b : A \multimap B}$$

$$(\multimap\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f \mid a \rangle : B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash f(a) : B}$$

$$(\multimap\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \rho x. b : A \multimap B}$$

$$(\multimap\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f \mid a \rangle : B}$$

$$(\multimap\text{-INTRO}) \frac{(x : A) \otimes \Gamma \vdash b : B}{\Gamma \vdash \partial x. b : A \multimap B}$$

Some rules

$$(\times\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma, \Delta \vdash a, b : A \times B}$$

$$(\otimes\text{-INTRO}) \frac{\Gamma \vdash a : A \quad \Delta \vdash b : B}{\Gamma \otimes \Delta \vdash a \otimes b : A \otimes B}$$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma, (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash f(a) : B}$$

$$(\multimap\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \rho x. b : A \multimap B}$$

$$(\multimap\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f \mid a \rangle : B}$$

$$(\multimap\text{-INTRO}) \frac{(x : A) \otimes \Gamma \vdash b : B}{\Gamma \vdash \partial x. b : A \multimap B}$$

$$(\multimap\text{-ELIM}) \frac{\Gamma \vdash a : A \quad \Delta \vdash f : A \multimap B}{\Gamma \otimes \Delta \vdash \langle a \mid f : B}$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle \quad f \longmapsto \partial x. \partial y. \langle x \otimes y \mid f$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle$$

$$f \longmapsto \lambda x. \lambda y. \langle x \otimes y \mid f$$

$$\lambda (x \otimes y). f \mid x \rangle \mid y \rangle \longleftarrow f$$

$$\lambda (x \otimes y). \langle x \mid \langle y \mid f \longleftarrow f$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle \quad f \longmapsto \partial x. \partial y. \langle x \otimes y \mid f$$

$$\lambda (x \otimes y). f \mid x \rangle \mid y \rangle \longleftarrow f \quad \partial (x \otimes y). \langle x \mid \langle y \mid f \longleftarrow f$$

$- \otimes Y$ is left adjoint to $Y \rightarrow -$

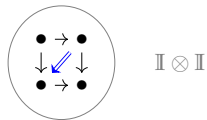
$Y \otimes -$ is left adjoint to $Y \multimap -$

hom-types

$$\mathrm{hom}_A(x, y) \equiv \sum_{f:\mathbb{I} \rightarrow A} (f \mid 0\rangle = x) \times (f \mid 1\rangle = y)$$

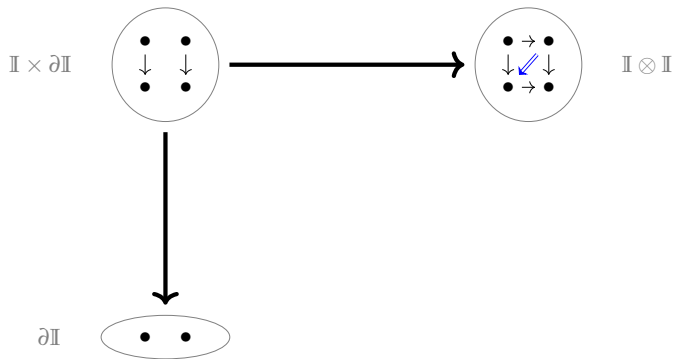
hom-types

$$\text{hom}_A(x, y) \quad \equiv \quad \sum_{f: \mathbb{I} \rightarrow A} (f \mid 0\rangle = x) \times (f \mid 1\rangle = y)$$



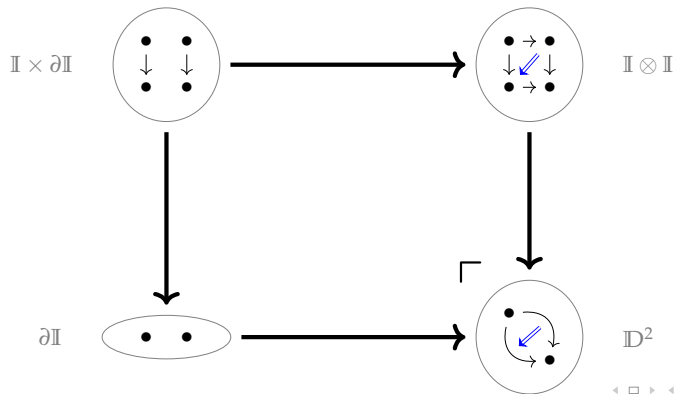
hom-types

$$\mathrm{hom}_A(x, y) \equiv \sum_{f: \mathbb{I} \rightarrow A} (f \mid 0\rangle = x) \times (f \mid 1\rangle = y)$$



hom-types

$$\mathrm{hom}_A(x, y) \equiv \sum_{f: \mathbb{I} \rightarrow A} (f \mid 0\rangle = x) \times (f \mid 1\rangle = y)$$



Categories

When does X behaves as a category ?

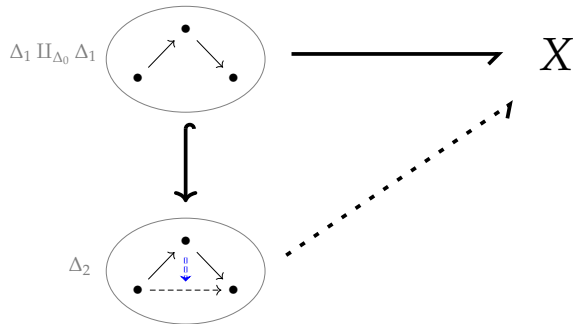
Categories

When does X behaves as a category ?

$$\Delta_1 \amalg_{\Delta_0} \Delta_1 \quad \begin{array}{c} \bullet \\ \nearrow \quad \searrow \\ \bullet \quad \bullet \end{array} \longrightarrow X$$

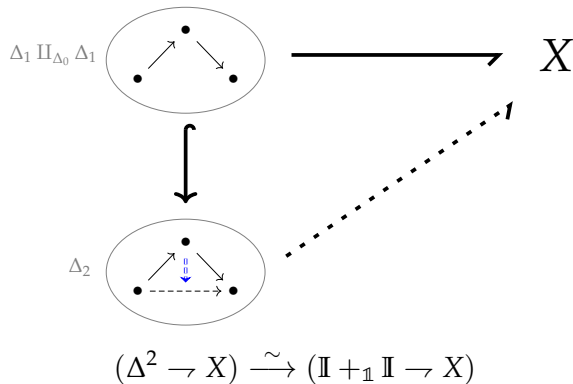
Categories

When does X behaves as a category ?



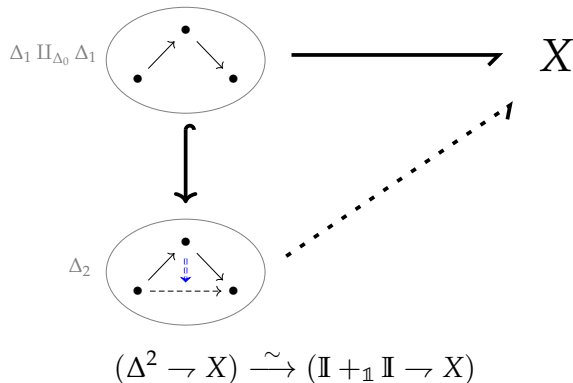
Categories

When does X behaves as a category ?



Categories

When does X behaves as a category ?



$$\sum_{x,y,z:X} \text{hom}_X(x, y) \times \text{hom}_X(y, z) \xrightarrow{\circ} \text{hom}_X(x, z)$$

batt: a type-checker for BaTT

 $/\backslash^{\cdot} _ \cdot^{\cdot} / \backslash$

- Ongoing Ocaml implementation by S. Mimram:
`github.com/smimram/batt`

batt: a type-checker for BaTT

/\ ^ . _ . ^ /\

- Ongoing Ocaml implementation by S. Mimram:
`github.com/smimram/batt`
- Lexer/parser, Type-checker, Metavariables, Unification.

batt: a type-checker for BaTT

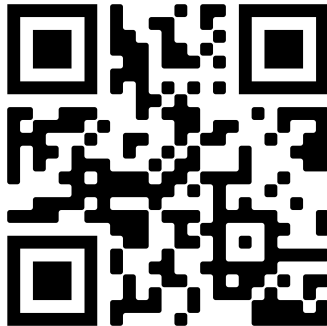
 $/\backslash^{\cdot} _ \cdot^{\cdot} / \backslash$

- Ongoing Ocaml implementation by S. Mimram:
`github.com/smimram/batt`
- Lexer/parser, Type-checker, Metavariables, Unification.
- Small module system, Holes à la Agda, Readable error messages !

Homotopy type theory ○○○○○○○○○	Simplicial type theory ○○○○○	The problem of hom types ○○	The bunched solution ○○○○○	From theory to application ●○○○
<pre>13) x-11)) (((?13927 x-0) x-1) x-2)))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?21559 x-2) x-10)) (((postulate54 (λ(x12×x13).(x-2 2 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((?21576 x-2) x-10)) ((λ(x12,x13).((λ(x14 x15).x-14) x-13)) x-2)),((J(((J(refl) ((?21450 x-2) x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate3) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?21458 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-15) x-13)) x-2))) x-11))) fal) ((J((x-9 (((postulate53 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) false))) ((?209 3 x-2) x-9)) ((λ(x10,x11).((λ(x12,x13).x-12) x-11)) x-2))).(((((((postulate100 (λ(x11×x12).(x-11 (Bool_ind(postulate36,postu ate37) x-12)))) (λ(x11×x12).x-12)) ((λ(x11,x12).x-11) (((?13927 x-0) x-1) x-2))) (λ(x11×x12).(((λ(x13,x14).x-13) x-2) (((postu ate52 (λ(x13×x14).(x-13 (Bool_ind(postulate36,postulate37) x-14)))) (λ(x13×x14).x-14)) (x-11 x-12)))))) Bool_ind(((λ(x11,x1).((λ(x13,x14).x-13) x-12)) (((?13927 x-0) x-1) x-2)),((λ(x11,x12).((λ(x13,x14).x-14) x-12)) (((?13927 x-0) x-1) x-2))) (λ(x1 ×x12).(Bool_ind(((J(((J(refl) ((?21189 x-2) x-11)) (((postulate54 (λ(x13×x14).(x-13 (Bool_ind(postulate36,postulate37) x-14))) (λ(x13×x14).x-14)) (x-11 false)))) ((?21206 x-2) x-11)) ((λ(x13,x14).((λ(x15,x16).x-15) x-14)) x-2)),((J(((J(refl) ((?210 0 x-2) x-11)) (((postulate54 (λ(x13×x14).(x-13 (Bool_ind(postulate36,postulate37) x-14)))) (λ(x13×x14).x-14)) (x-11 true)))) ((?21088 x-2) x-11)) ((λ(x13,x14).((λ(x15,x16).x-16) x-14)) x-2))) x-12))) true) ((J((x-9 (((postulate53 (λ(x11×x12).(x-11 (Bool_ind(postulate36,postulate37) x-12)))) (λ(x11×x12).x-12)) true))) (((?20967 x-2) x-9) x-10)) ((λ(x11,x12).((λ(x13,x14) -14) x-12)) x-2)))) HOLE in file ./hom.batt line 84 characters 1-2 : (Σ(x9 : (x9 : (?13995 x-2)) → (((((((postulate98 (λ(x10×x11).(x-10 (Bool_in (postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10,x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13). -12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) ool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?23045 x-2) x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36 postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((?23062 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),(J(((J(refl) ((?22936 x-2) x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x- 3)) (x-10 true)))) ((?22944 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-15) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x10,x11).x-10) x-2 x-9)).(Σ(x10 : (((((((postulate100 (λ(x10×x11).(x-10 (Bool_ind(postulate36,postulate37) x-11)))) (λ(x10×x11).x-11)) ((λ(x10 x11).x-10) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(((λ(x12,x13).x-12) x-2) (((postulate52 (λ(x12×x13).(x-12 (Bool_ind(postul te36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 x-11)))))) Bool_ind(((λ(x10,x11).((λ(x12,x13).x-12) x-11)) (((?13927 x-0) x-1) x-2)),((λ(x10,x11).((λ(x12,x13).x-13) x-11)) (((?13927 x-0) x-1) x-2))) (λ(x10×x11).(Bool_ind(((J(((J(refl) ((?22676 x-2 x-10)) (((postulate54 (λ(x12×x13).(x-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 false)))) ((22693 x-2) x-10)) ((λ(x12,x13).((λ(x14,x15).x-14) x-13)) x-2)),((J(((J(refl) ((?22567 x-2) x-10)) (((postulate54 (λ(x12×x13).(-12 (Bool_ind(postulate36,postulate37) x-13)))) (λ(x12×x13).x-13)) (x-10 true)))) ((?22575 x-2) x-10)) ((λ(x12,x13).((λ(x1 13) x-13)) x-2))) x-11))) x-9) (((λ(x</pre>				

Examples

smimram.github.io/batt/



The end

Thank you ! Questions ?

